

MANUALE TECNICO – CinemaX

Sistema di Gestione Proiezioni e Prenotazioni Cinematografiche

Autore: [Demontis Tommaso, Piantalunga Diego, Proserpio Fabio]

Corso: Laboratorio Interdisciplinare A *Dipartimento di Scienze Teoriche e Applicate –
Università degli Studi dell’Insubria

Versione documento: 1.2

Data: [09/06/2026]

Indice

1. Introduzione Tecnica
2. Requisiti Tecnici:
 - 2.1 Requisiti hardware
 - 2.2 Requisiti software
 - 2.3 Requisiti dei file
 - 2.4 Permessi e accessi ai file
 - 2.5 Compatibilità
3. Architettura del sistema
4. Diagramma delle Classi
5. Relazioni tra classi
6. Scelte architettureali
7. Strutture dati utilizzate
 - 7.1 Vettori
 - 7.2 ArrayList
8. Struttura file
 - 8.1 Utenti.txt
 - 8.2 Prenotazioni.txt
 - 8.3 Film.txt
 - 8.4 Proiezioni.csv
9. Scelte algoritmiche
 - 9.1 Cifratura di Password
 - 9.2 Decifratura di Password

- 9.3 Cambio Data prenotazione
- 9.4 Ricerca Proiezioni Disponibili

- 10. Sitografia e riferimenti
- 11. Credenziali di test

1. Introduzione Tecnica

CinemaX è un'applicazione software sviluppata per la gestione operativa di un cinema, che consente l'amministrazione centralizzata di proiezioni cinematografiche e prenotazioni.

Il sistema è basato su un'interfaccia testuale, organizzata tramite menu strutturati, che garantisce un accesso controllato alle funzionalità in base ai diversi livelli di autorizzazione definiti.

L'architettura prevede quattro tipologie di utenti, ciascuna associata a specifici permessi operativi:

- **Guest:** accesso in sola lettura alle proiezioni disponibili.
- **Cliente:** gestione delle prenotazioni personali, includendo operazioni di creazione, visualizzazione, modifica ed eliminazione; il sistema supporta inoltre le funzionalità di registrazione e autenticazione.
- **Proiezionista:** gestione delle entità film e proiezioni, con operazioni di inserimento, aggiornamento ed eliminazione.
- **Bigliettaio:** consultazione e verifica delle prenotazioni registrate nel sistema.

Il presente documento, destinato a un pubblico tecnico, descrive le scelte progettuali adottate, gli algoritmi implementati, le strutture dati utilizzate, l'organizzazione dei file e le principali decisioni architetturali che caratterizzano il sistema CinemaX.

2. Requisiti Tecnici

Questa sezione descrive in modo dettagliato tutti i requisiti necessari per installare, configurare ed eseguire correttamente il software *CinemaX*.

I requisiti sono suddivisi in **requisiti hardware**, **requisiti software** e **requisiti dei file**.

2.1 Requisiti Hardware

CinemaX è un'applicazione leggera e può essere eseguita sulla maggior parte dei computer moderni.

Sono sufficienti:

- **Processore:** qualsiasi CPU dual-core o superiore
- **RAM:** almeno 4 GB

- **Spazio su disco:** minimo 50 MB per il programma e i file di dati
- **Monitor:** risoluzione minima 1280×720 (per una corretta visualizzazione del terminale/IDE)

Non sono richieste schede grafiche dedicate o componenti avanzati.

2.2 Requisiti Software

Per eseguire CinemaX è necessario che il sistema disponga dei seguenti componenti:

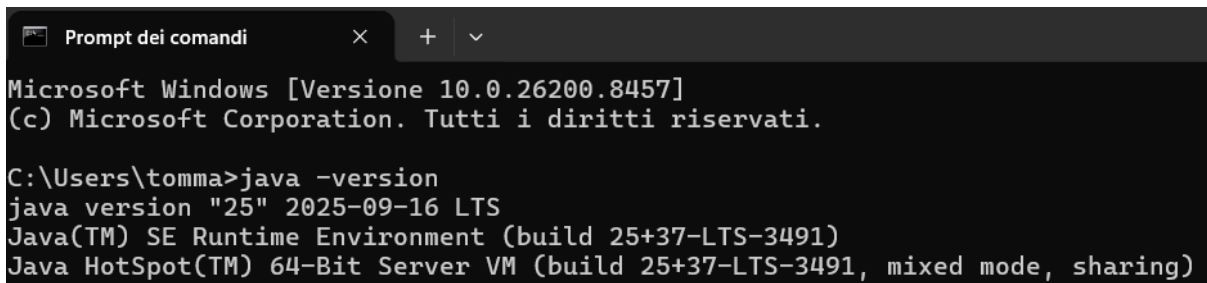
Java Development Kit (JDK) o Java Runtime Environment (JRE)

- Versione **17** o superiore
- Compatibile con Windows, macOS e Linux
- Per verificare che Java sia installato correttamente, è necessario eseguire il seguente comando **nel terminale del proprio sistema operativo**:

Windows

Aprire **Prompt dei comandi**

- (Start → digitare “cmd” → Invio)
- Digitare: `java -version`



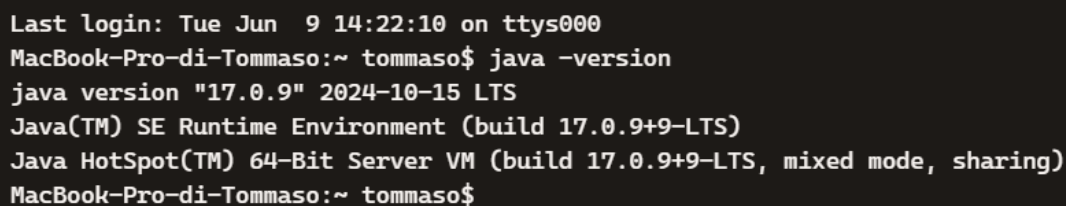
```
Prompt dei comandi
Microsoft Windows [Versione 10.0.26200.8457]
(c) Microsoft Corporation. Tutti i diritti riservati.

C:\Users\tomma>java -version
java version "25" 2025-09-16 LTS
Java(TM) SE Runtime Environment (build 25+37-LTS-3491)
Java HotSpot(TM) 64-Bit Server VM (build 25+37-LTS-3491, mixed mode, sharing)
```

macOS

Aprire **Terminale**

- (Applicazioni → Utility → Terminale)
- Digitare: `java -version`



```
Last login: Tue Jun  9 14:22:10 on ttys000
MacBook-Pro-di-Tommaso:~ tommaso$ java -version
java version "17.0.9" 2024-10-15 LTS
Java(TM) SE Runtime Environment (build 17.0.9+9-LTS)
Java HotSpot(TM) 64-Bit Server VM (build 17.0.9+9-LTS, mixed mode, sharing)
MacBook-Pro-di-Tommaso:~ tommaso$
```

Linux

- Aprire il **Terminale**
- Digitare: `java -version`

```
tommaso@ubuntu:~$ java -version
openjdk version "17.0.9" 2024-10-15
OpenJDK Runtime Environment (build 17.0.9+9-Ubuntu-1ubuntu1)
OpenJDK 64-Bit Server VM (build 17.0.9+9-Ubuntu-1ubuntu1, mixed mode, sharing)
tommaso@ubuntu:~$
```

Se Java è installato correttamente, il sistema mostrerà una risposta simile a:

```
C:\Users\Tommaso>java -version
java version "17.0.9" 2024-10-15 LTS
Java(TM) SE Runtime Environment (build 17.0.9+9-LTS)
Java HotSpot(TM) 64-Bit Server VM (build 17.0.9+9-LTS, mixed mode, sharing)

✓ Java è installata correttamente sul sistema.
C:\Users\Tommaso>
```

Se invece il comando non viene riconosciuto, significa che Java non è installato o non è configurato nel PATH.

Sistema Operativo Supportato

- **Windows 10/11**
- **macOS 10.15 o superiore**
- **Distribuzioni Linux** (Ubuntu, Debian, Fedora, ecc.)

Ambiente di esecuzione

È possibile utilizzare:

- un **IDE** (IntelliJ IDEA, Eclipse, NetBeans)
- oppure un **terminale**/prompt dei comandi

Non è richiesta alcuna installazione aggiuntiva oltre a Java.

2.3 Requisiti dei File

CinemaX utilizza file di testo per memorizzare film, proiezioni e prenotazioni.

Per funzionare correttamente, il programma richiede la presenza della cartella: `/data` all'interno della directory principale del progetto.

File necessari

All'interno della cartella devono essere presenti:

Film.txt

Contiene l'elenco dei film disponibili nel sistema.

- Ogni riga rappresenta un film con i relativi campi (titolo, genere, regista, anno, durata, età minima).

proiezioni.csv

Contiene tutte le proiezioni programmate.

- Ogni riga contiene data, ora, film associato e costo del biglietto.

prenotazioni.csv (opzionale)

- Viene generato automaticamente quando un cliente effettua la prima prenotazione.

Formattazione corretta dei file

È fondamentale che i file rispettino la formattazione prevista, ad esempio:

Film.txt

Titolo,Genere,Regista,Anno,Durata,EtaMinima

Esempio: Avatar,Action,Cameron,2009,162,12

```
titolo,genere,regista,anno,durata,etaMinima
The Dark Knight,Action,Christopher Nolan,2008,152,13
Gladiator,Action,Ridley Scott,2000,155,16
Mad Max: Fury Road,Action,George Miller,2015,120,16
John Wick,Action,Chad Stahelski,2014,101,18
Die Hard,Action,John McTiernan,1988,132,16
Mission Impossible Fallout,Action,Christopher McQuarrie,2018,147,13
Top Gun Maverick,Action,Joseph Kosinski,2022,131,10
The Avengers,Action,Joss Whedon,2012,143,10
Casino Royale,Action,Martin Campbell,2006,144,13
The Batman,Action,Matt Reeves,2022,176,14
Indiana Jones and the Last Crusade,Adventure,Steven Spielberg,1989,127,10
Jurassic Park,Adventure,Steven Spielberg,1993,127,10
Jumanji,Adventure,Joe Johnston,1995,104,6
Pirates of the Caribbean,Adventure,Gore Verbinski,2003,143,10
The Revenant,Adventure,Alejandro González Iñárritu,2015,156,16
Life of Pi,Adventure,Ang Lee,2012,127,10
King Kong,Adventure,Peter Jackson,2005,187,13
The Mummy,Adventure,Stephen Sommers,1999,124,10
Uncharted,Adventure,Ruben Fleischer,2022,116,10
Journey to the Center of the Earth,Adventure,Eric Brevig,2008,93,6
Toy Story,Animation,John Lasseter,1995,81,3
Finding Nemo,Animation,Andrew Stanton,2003,100,3
Frozen,Animation,Chris Buck,2013,102,3
Shrek,Animation,Andrew Adamson,2001,90,6
Coco,Animation,Lee Unkrich,2017,105,6
Up,Animation,Pete Docter,2009,96,3
Cars,Animation,John Lasseter,2006,117,3
Inside Out,Animation,Pete Docter,2015,95,3
Ratatouille,Animation,Brad Bird,2007,111,3
```

proiezioni.csv

"2025-05-12 20:30:00","Avatar",Action,"Cameron",2009,162,12,8.50

```
data,genere,regista,anno,durata,eta,costo
"2027-12-30 10:30","A Beautiful Mind",Biography,"Ron Howard",2001,135,12,8.5
"2027-12-29 16:00","Good Will Hunting",Drama,"Gus Van Sant",1997,126,0,8.5
"2027-12-29 12:30","The Best Years of Our Lives",Drama,"William Wyler",1946,170,12,8.5
"2027-12-29 10:00","To Have and Have Not",Adventure,"Howard Hawks",1944,100,6,8.0
"2027-12-28 15:30","Blue Velvet",Drama,"David Lynch",1986,120,12,7.5
"2027-12-28 13:00","The Philadelphia Story",Comedy,"George Cukor",1940,112,12,8.5
"2027-12-28 10:00","Good Will Hunting",Drama,"Gus Van Sant",1997,126,0,6.5
"2027-12-27 14:00","The Blues Brothers",Action,"John Landis",1980,133,0,8.5
"2027-12-27 10:30","The Bridge on the River Kwai",Adventure,"David Lean",1957,161,6,7.5
"2027-12-26 19:30","Good Will Hunting",Drama,"Gus Van Sant",1997,126,0,10.0
"2027-12-26 16:00","The Bridge on the River Kwai",Adventure,"David Lean",1957,161,6,8.5
"2027-12-26 13:00","Good Will Hunting",Drama,"Gus Van Sant",1997,126,0,8.5
"2027-12-26 10:00","Blue Velvet",Drama,"David Lynch",1986,120,12,7.5
"2027-12-25 22:00","The Best Years of Our Lives",Drama,"William Wyler",1946,170,12,10.0
"2027-12-25 18:00","Gandhi",Biography,"Richard Attenborough",1982,191,0,9.0
"2027-12-25 14:30","The Lord of the Rings: The Fellowship of the Ring",Action,"Peter Jackson",2001,178,0,7.0
"2027-12-25 10:30","The Lord of the Rings: The Fellowship of the Ring",Action,"Peter Jackson",2001,178,0,8.0
"2027-12-24 13:30","Misery",Drama,"Rob Reiner",1990,107,16,8.0
"2027-12-24 10:30","The Treasure of the Sierra Madre",Adventure,"John Huston",1948,126,12,8.5
"2027-12-23 10:00","The Hurricane",Biography,"Norman Jewison",1999,146,16,8.5
"2027-12-22 17:00","C.R.A.Z.Y.",Comedy,"Jean-Marc Vallée",2005,129,12,8.0
"2027-12-22 14:30","Misery",Drama,"Rob Reiner",1990,107,16,7.5
"2027-12-22 10:30","Veer-Zaara",Drama,"Yash Chopra",2004,192,0,7.5
"2027-12-21 16:00","Primal Fear",Crime,"Gregory Hoblit",1996,129,12,7.0
"2027-12-21 13:00","The Blues Brothers",Action,"John Landis",1980,133,0,8.0
```

Errori di formattazione possono impedire al programma di leggere correttamente i dati.

2.4 Permessi e Accesso ai File

Il programma deve poter:

- leggere i file nella cartella /data
- scrivere nei file quando vengono aggiunte proiezioni, film o prenotazioni

Assicurarsi che:

- la cartella non sia protetta da permessi di sola lettura
- l'utente abbia i permessi di scrittura nella directory del progetto

2.5 Compatibilità

CinemaX è compatibile con:

- terminali standard
- emulatori di console
- IDE con console integrata

Non richiede connessione Internet né componenti esterni.

3. Architettura del Sistema

La struttura del progetto è stata sviluppata utilizzando il linguaggio Java, basato sul paradigma della programmazione orientata agli oggetti (OOP – Object Oriented Programming). Questo approccio consente di modellare il sistema attraverso l'utilizzo di classi e oggetti, permettendo una migliore organizzazione del codice, una maggiore modularità e una gestione più efficiente delle funzionalità del programma.

All'interno della cartella src sono presenti tutte le classi utilizzate per la realizzazione del progetto, ognuna delle quali rappresenta una specifica entità o una particolare funzionalità del sistema.

Le principali classi implementate sono le seguenti:

- **Cinemax:** rappresenta la classe principale (Main) del progetto. Contiene il punto di avvio del programma e gestisce l'esecuzione generale dell'applicazione.
- **Film:** classe utilizzata per creare oggetti che rappresentano i film presenti nel sistema. Ogni oggetto di tipo Film contiene informazioni relative al titolo, alla durata, al genere e ad altri dati utili alla gestione cinematografica.
- **Proiezione:** rappresenta le diverse proiezioni cinematografiche inserite dai proiezionisti. Ogni oggetto di questa classe contiene riferimenti a un oggetto di tipo Film, oltre alle informazioni relative alla data, all'orario e alla sala della proiezione.
- **Utente:** classe padre del sistema, utilizzata per rappresentare genericamente tutti gli utenti. Da questa classe derivano diverse sottoclassi che ereditano attributi e metodi comuni, applicando il principio di ereditarietà tipico della programmazione orientata agli oggetti.
- **Guest:** tipologia di utente non autenticato all'interno del sistema. Può accedere solamente a funzionalità limitate, come la visualizzazione delle proiezioni disponibili.
- **Cliente:** utente autenticato tramite credenziali personali. Rispetto al Guest, dispone di funzionalità aggiuntive, come la possibilità di effettuare prenotazioni e gestire le proprie operazioni all'interno del sistema.
- **Proiezionista:** utente autenticato con permessi specifici dedicati alla gestione delle proiezioni cinematografiche. Può inserire, modificare o eliminare le proiezioni presenti nel sistema.
- **Bigliettaio:** utente autenticato che dispone di funzionalità dedicate alla gestione dei biglietti e delle prenotazioni effettuate dai clienti.
- **Prenotazione:** classe che permette la creazione di oggetti di tipo prenotazione. Ogni prenotazione contiene le informazioni relative alla proiezione scelta dal cliente e ai dati necessari per confermare la prenotazione stessa.

- **Inserimenti:** classe statica utilizzata per gestire tutti gli inserimenti da tastiera effettuati dagli utenti. Consente di centralizzare e controllare la validazione dei dati inseriti nel sistema.
- **Menu:** classe che contiene tutti i menu di navigazione disponibili per le diverse tipologie di utenti. Permette di organizzare in maniera chiara e ordinata le operazioni eseguibili nel programma.

Per garantire il corretto funzionamento del progetto è stato inoltre necessario utilizzare diversi file esterni, impiegati come vere e proprie banche dati del sistema. L'utilizzo dei file consente di mantenere salvate le informazioni anche dopo la chiusura del programma, permettendo così la persistenza dei dati tra un'esecuzione e l'altra.

All'interno dei file vengono memorizzate informazioni relative agli utenti registrati, ai film presenti nel catalogo, alle proiezioni disponibili, alle prenotazioni effettuate e a tutti i dati necessari alla gestione generale del sistema.

Per la gestione dei dati si è deciso di utilizzare diverse strutture dati, in particolare vettori (array) e liste (ArrayList). La struttura maggiormente utilizzata è stata la lista dinamica (ArrayList), poiché offre una gestione più flessibile degli elementi rispetto ai vettori tradizionali. Le liste permettono infatti di aggiungere, rimuovere e modificare dati dinamicamente durante l'esecuzione del programma, senza dover definire preventivamente una dimensione fissa della struttura. Questo approccio risulta particolarmente utile in un progetto in cui il numero di utenti, prenotazioni o proiezioni può variare continuamente.

4. Diagramma delle Classi

Classe Film

Attributi

- titolo : String
- genere : String
- regista : String
- anno : int
- durata : int
- etaMinima : int

Metodi

- Film(String titolo,String genere,String regista, int anno, int durata, int etaMinima) costruttore
- getTitolo()
- getGenere()
- getRegista()

- getAnno()
- getDurata()
- getEtaMinima()
- SetTitolo(String titolo)
- SetGenere(String genere)
- SetRegista(String regista)
- SetAnno(int anno)
- SetDurata(int durata)
- SetEtaMinima(int etaMinima)
- toString()

Classe Prenotazione

Attributi

- id : int
- utente : Cliente
- proiezione : Proiezione
- postiPrenotati : int
- scaduta : boolean

Metodi

- Prenotazione(Cliente utente,Proiezione proiezione, int postiPrenotati, boolean scaduta) costruttore
- Prenotazione(int id,Cliente utente,Proiezione proiezione, int postiPrenotati, boolean scaduta) costruttore
- getId()
- getCliente()
- getProiezione()
- getPostiPrenotati()
- setPostiPrenotati(int postiPrenotati)
- getScaduta()
- getCostoTotale()
- InseriscId()
- VisualizzaPrenotazione()

Classe Cliente

Attributi

- Ereditati da Utente

Metodi

- Cliente(String nome,String cognome,String username,String password,LocalDate nascita,String domicilio)

- InserisciPrenotazione(Proiezione p)
- LeggiPrenotazioni Personali()
- modificaPrenotazione()
- eliminaPrenotazione()
- ProiezioniDisponibili()
- PostiGiaPrenotati(Proiezione p)
- NuovaData(Proiezione p, int posti, Proiezione nuova)
- RicercaPrenotazione(Prenotazione p, ArrayList<Prenotazione> tutte)

Classe Bigliettaio

Attributi

- Ereditati da Utente

Metodi

- Bigliettaio(String nome,String cognome,String username,String password,LocalDate nascita,String domicilio) costruttore
- RicercaPrenotazione(int id, ArrayList<Prenotazione> p)
- RicercaPrenotazione(String nome,String cognome, ArrayList<Prenotazione> p)
- RicercaPrenotazione(LocalDate data, ArrayList<Prenotazione> p)
- RicercaPrenotazione(LocalDate data1,LocalDate data 2, ArrayList<Prenotazione p>
- RicercaPrenotazioneCombinata()
- ScegliPrenotazione(ArrayList<Prenotazione> p)
- LeggiPrenotazioni()
- SelezionaCliente(String nome,String cognome,String username,String password)
- SelezionaProiezione(LocalDate data, LocalTime ora,String titolo)
- CheckScaduta(LocalDate data, LocalTime ora)

Classe Menu

Attributi

- Nessuno

Metodi

- mostraMenuIniziale()
- mostraMenuGuest()
- mostraMenuCliente()
- mostraMenuProiezionista()
- mostraMenuBigliettaio()
- mostraMenuCercaProiezione()
- sceltaBigliettaioRicerca()
- MenuSceltaGenere()
- MenuRicercaData()

- MenuRicercaPrezzo()
- MenuModificaPrenotazione()

Classe Inserimenti

Attributi

- Nessuno

Metodi

- inserisciTitolo(Scanner input)
- inserisciGenere(Scanner input)
- inserisciRegista(Scanner input)
- inserisciAnno(Scanner input)
- inserisciDurata(Scanner input)
- inserisciEtaMinima(Scanner input)
- inserisciOra(Scanner input)
- inserisciCosto(Scanner input)
- inserisciNome(Scanner input)
- inserisciCognome(Scanner input)
- inserisciUsername(Scanner input)
- inserisciPassword(Scanner input)
- inserisciDomicilio(Scanner input)
- inserisciData(Scanner input)
- InserisciPosti(Scanner input)

Classe Guest

Attributi

- chiave : char

Metodi

- Guest() costruttore
- Registrazione()
- Login()
- ScriviFile(String nome,String cognome,String username,String password,LocalData nascita, String luogo,String ruolo)
- LeggiFile()
- EncodedPsw(String password)
- DecodePsw(String password)
- CheckUsername(String username)

Classe Cinemax

Attributi

- menu : Menu
- g : Guest
- ut : Utente
- scelta : int
- uscita : boolean

Metodi

- main(String[] args)

Classe Proiezione

Attributi

- data : LocalDate
- ora : LocalTime
- film : Film
- costo : float

Metodi

- Proiezione(LocalDate data,LocalTime ora,Film film,float costo)
- GetData()
- GetOra()
- GetFilm()
- GetCosto()
- SetData(LocalDate data)
- SetOra(LocalTime ora)
- SetFilm()
- SetCosto()
- toString()

Classe Proiezionista

Attributi

- Ereditati da Utente

Metodi

- Proiezionista(String nome, String cognome, String username, String password, LocalDate nascita, String domicilio) costruttore
- InserisciFilm()
- InserisciProiezione(Film film)
- modificaProiezione()
- eliminaProiezione()
- selezionaFilm(ArrayList<Film> film)
- leggiFilm()
- ControllaFilm(String Titolo)

Classe Utente

Attributi

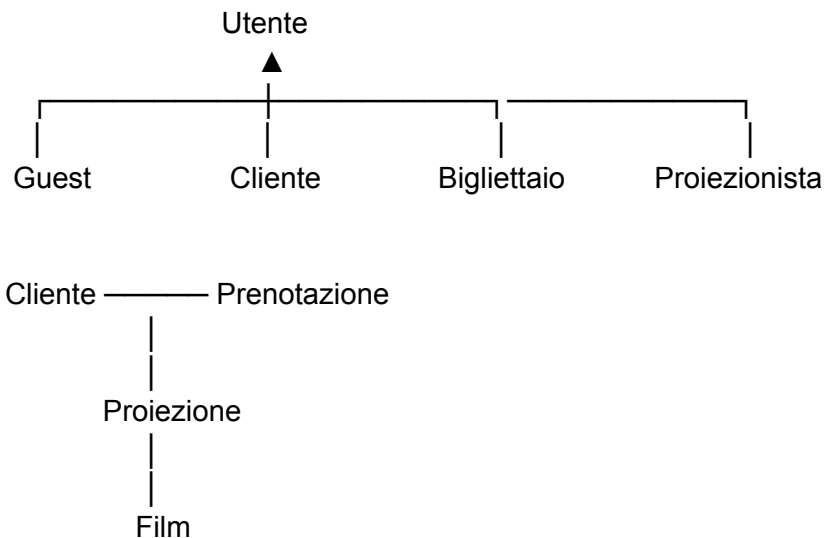
- nome : String
- cognome : String
- username : String
- password : String
- nascita : LocalDate
- domicilio : String
- mansione : String
- chiave : char

Metodi

- Utente(String nome, String cognome,String username,String password,LocalDate nascita,String domicilio,String mansione) costruttore
- Utente(String nome,String cognome,String username,String password,String domicilio,String mansione) costruttore
- Utente() costruttore
- SetNome(String nome)
- SetCognome(String cognome)
- SetUsername(String username)
- SetPassword(String password)
- SetNascita(LocalDate nascita)
- SetDomicilio(String domicilio)
- GetNome()
- GetCognome()
- GetUsername()
- GetPassword()
- GetNascita()

- GetAnni()
- GetDomicilio()
- GetMansione()
- SelezioneProiezione(ArrayList<Proiezione>p)
- leggiProiezioni()
- CercaProiezione()
- RicercaPerTitolo(String titolo,ArrayList<Proiezione> p)
- RicercaPerData(LocalDate data,ArrayList<Proiezione> p)
- RicercaPerData(LocalDate data1,LocalDate data2,ArrayList<Proiezione> p)
- RicercaPerGenere(String genere,ArrayList<Proiezione> p)
- RicercaPerPrezzo(float prezzo,ArrayList<Proiezione> p)
- RicercaPerPrezzo(float prezzo1,float prezzo2,ArrayList<Proiezione> p)
- RicercaCombinata(ArrayList<Proiezione> p)
- CheckString(String ins, String obj)
- toString()

5. Relazioni tra Classi



6. Scelte Architettureali

La classe Utente è stata definita come astratta poiché rappresenta una classe padre generica dalla quale ereditano le classi Guest, Client, Bigliettaio e Proiezionista. Questa scelta progettuale permette di raccogliere all'interno di Utente attributi e metodi comuni a tutte le tipologie di utenti, evitando duplicazione di codice e favorendo il principio di ereditarietà tipico della programmazione object-oriented. Inoltre, l'utilizzo di una classe

astratta consente di definire una struttura comune mantenendo la possibilità di specializzare comportamenti specifici nelle sottoclassi.

La classe Inserimenti è stata progettata come classe statica, infatti tutti i metodi al suo interno sono dichiarati static.

Questa decisione è stata presa per facilitare la riutilizzazione del codice: i metodi di inserimento dei dati vengono infatti utilizzati da numerose altre classi del progetto. Rendendo tali metodi statici, è possibile richiamarli direttamente senza creare istanze della classe, migliorando semplicità di utilizzo, organizzazione e modularità del codice.

Per quanto riguarda la gestione dei dati, si è scelto di utilizzare i file come strumento di memorizzazione delle informazioni.

Questa soluzione è stata adottata sia per la semplicità di implementazione sia per le conoscenze ancora introduttive riguardo ai database relazionali, trattati nel percorso universitario solo successivamente. Nonostante ciò, l'utilizzo dei file permette comunque una gestione persistente dei dati, risultando adeguato agli obiettivi del progetto e coerente con il livello di apprendimento raggiunto durante lo sviluppo.

La scelta del linguaggio Java è motivata principalmente dall'efficacia del paradigma object-oriented offerto dal linguaggio stesso.

Java consente infatti una gestione chiara di concetti fondamentali come classi, oggetti, ereditarietà, incapsulamento e polimorfismo, elementi centrali nello sviluppo del progetto. Inoltre, essendo il linguaggio studiato durante il primo semestre del primo anno universitario nel corso di Programmazione, ha rappresentato la soluzione più adatta per applicare concretamente le competenze apprese.

All'interno delle varie classi sono stati utilizzati anche diversi metodi statici.

Questa scelta è stata effettuata sia per permettere il richiamo diretto di funzionalità interne senza dover creare oggetti inutili, sia per favorire il riutilizzo di determinate operazioni da parte di altre classi del sistema. In questo modo il codice risulta più organizzato, modulare e facilmente mantenibile.

7. Strutture Dati Utilizzate

7.1 Vettori

Nel progetto l'utilizzo dei vettori è stato limitato ai casi in cui la dimensione dei dati fosse nota a priori. In particolare, sono stati impiegati durante la lettura e l'elaborazione delle righe provenienti dai file di testo e CSV. Le funzioni `split()` restituiscono infatti array di stringhe (`String[]`) utilizzati per suddividere una riga nei suoi campi costitutivi, come data, ora e informazioni relative a utenti, proiezioni e prenotazioni. Ad esempio, durante la lettura delle proiezioni vengono creati i vettori campi, dataOra, datiD e datiO, la cui dimensione è nota perché dipende dal formato fisso del file.

La scelta del vettore è stata effettuata poiché il numero di elementi da memorizzare è prefissato e non necessita di operazioni di inserimento o rimozione dinamica. L'utilizzo degli array consente quindi una gestione semplice ed efficiente dei dati.. Analogamente, anche

nella lettura delle prenotazioni vengono utilizzati vettori di stringhe ottenuti tramite `split()` per separare i campi della riga letta.

7.2 ArrayList

La struttura dati maggiormente utilizzata nel progetto è l'ArrayList, scelta quando il numero di elementi non è noto a priori oppure può variare durante l'esecuzione del programma. Le liste dinamiche vengono utilizzate per memorizzare utenti, proiezioni, prenotazioni e risultati delle ricerche. Ad esempio, i metodi `LeggiFile()`, `leggiProiezioni()` e `LeggiPrenotazioni()` ricostruiscono i dati presenti nei file aggiungendo progressivamente gli oggetti letti all'interno di una ArrayList.

L'ArrayList è stata inoltre utilizzata per gestire i risultati filtrati delle ricerche, le prenotazioni personali dei clienti, le proiezioni disponibili e le selezioni effettuate dai vari utenti del sistema. In questi casi non è possibile conoscere in anticipo quanti elementi soddisferanno i criteri di ricerca o quanti record saranno presenti nei file; per questo motivo una struttura dinamica risulta più adatta rispetto a un vettore tradizionale, di cui l'implementazione nel progetto in questi casi risulta inefficace.

8. Struttura dei File

Per poter mantenere i dati durante le continue esecuzioni del programma da parte degli utenti, si è deciso di adottare una struttura basata sui file, così da consentire operazioni di aggiornamento, modifica ed eliminazione delle informazioni memorizzate.

Grazie all'utilizzo di file di testo e di file in formato CSV, è possibile eseguire diverse operazioni sui dati, come:

- inserimento di nuovi record;
- modifica degli elementi già presenti;
- eliminazione di dati preesistenti.

Per operare correttamente sui file è necessario conoscere la loro posizione all'interno della cartella del progetto. Infatti, nel caso in cui la JVM non riuscisse a trovare uno dei file richiesti, verrebbe generata un'eccezione che interromperebbe l'esecuzione del programma.

Nel progetto Cinemax si è scelto di utilizzare quattro file: tre file di testo e un file CSV fornito in dotazione.

I file utilizzati sono:

- Utenti.txt
- Prenotazioni.txt
- Film.txt
- proiezioni.csv

Tutti i file si trovano all'interno della cartella data del progetto Cinemax.

8.1 Utenti.txt

```
1 nome,cognome,username,password,giorno,mese,anno,domicilio,ruolo
2 Alice,Proserpio,Alix2008,☼0T0A,9,11,2008,Carate Urio,Cliente
3 Andrea,Proserpio,andrux32,0A00,11,2,1975,CarateUrio,Cliente
4 Alberto,Demontis,Albericoxx,☼000,12,12,1995,Cernobbio,Cliente
5 Noemi,Ancora,noem21,☼0f0,4,12,1996,Pero,Cliente
6 Emma,Gandola,gando06,000f00,8,7,2006,Lezzeno,Cliente
7 Tommaso,Demontis,tommydemo,☼0f000,18,5,2006,Cernobbio,Cliente
8 Mario,Albenga,proiezionista1,0f00/0000,12,9,2000,Como,Proiezionista
9 Luigi,Castaldo,proiezionista2,0000/0000,30,1,1975,Como,Proiezionista
10 Marisa,Caldi,bigliettaio1,0f00/0000,7,7,2003,Cernobbio,Bigliettaio
11 Eleonora,Bussoli,bigliettaio2,0000/0000,21,6,1967,Carate Urio,Bigliettaio
12 Enrico,Vitta,bigliettaio3,☼0T0A,20,4,1955,Menaggio,Bigliettaio
13 Arianna,Dangelo,bigliettaio4,0000000000000,1,12,2001,Moltrasio,Bigliettaio
14 Alessandro,Blanco,bigliettaio5,00000000,12,8,1993,Arcore,Bigliettaio
15 Fabio,Proserpio,fabietto06,☼0T0A,7,7,2006,Carate Urio,Cliente
```

Il file Utenti.txt rappresenta la banca dati dedicata al salvataggio e alla gestione degli utenti registrati al sistema.

Le informazioni memorizzate per ciascun utente sono le seguenti:

- nome;
- cognome;
- username;
- password;
- giorno di nascita;
- mese di nascita;
- anno di nascita;
- domicilio;
- luogo;
- ruolo dell'utente.

La data di nascita è stata suddivisa in giorno, mese e anno per semplificare il riassettaggio dell'oggetto LocalDate durante la fase di lettura dei dati degli utenti, in particolare quando viene invocato il metodo Login().

Per motivi di sicurezza, la password non viene salvata in chiaro, ma cifrata tramite un algoritmo basato su una chiave privata. In questo modo, le informazioni risultano illeggibili senza conoscere la chiave di decodifica.

È stato inoltre introdotto il concetto di ruolo dell'utente, così da distinguere le diverse categorie di entità che possono interagire con il programma. Nel file sono presenti

esclusivamente gli utenti registrati, mentre i guest non vengono salvati all'interno della struttura dati.

Questa scelta facilita la creazione degli oggetti utente in base al ruolo associato durante la fase di caricamento dei dati.

8.2 Prenotazioni.txt

```
ID, Nome, Cognome, Username, Password, data, ora, film, posti prenotati
2,Alice,Proserpio,Alix2008,👤0776943,2027-12-26,13:0:0,Good Will Hunting,4
1,Alice,Proserpio,Alix2008,👤0776943,2027-5-5,10:0:0,Kakushi-toride no san-akunin,3
2,Fabio,Proserpio,fabietto06,👤0776943,2027-9-14,10:0:0,Kal Ho Naa Ho,2
3,Fabio,Proserpio,fabietto06,👤0776943,2027-2-28,10:0:0,The Notebook,8
4,Fabio,Proserpio,fabietto06,👤0776943,2026-11-14,10:30:0,Majo no takkyûbin,7
```

Il file appena descritto mantiene in memoria le varie prenotazioni effettuate dagli utenti, tenendo traccia sia dell'utente che ha effettuato la prenotazione sia della proiezione selezionata.

Grazie a questa banca dati è possibile:

- aggiungere nuove prenotazioni;
- modificare prenotazioni esistenti;
- eliminare prenotazioni già registrate.

È stato inoltre introdotto il campo relativo ai posti prenotati, utile per conteggiare il numero totale di posti occupati per una determinata proiezione. In questo modo il sistema può verificare automaticamente la disponibilità residua della sala.

Al raggiungimento della soglia massima di 200 posti, che rappresenta la capienza massima della sala cinematografica, non è più possibile effettuare ulteriori prenotazioni.

8.3 Film.txt

```

titolo, genere, regista, anno, durata, etaMinima
The Dark Knight, Action, Christopher Nolan, 2008, 152, 13
Gladiator, Action, Ridley Scott, 2000, 155, 16
Mad Max: Fury Road, Action, George Miller, 2015, 120, 16
John Wick, Action, Chad Stahelski, 2014, 101, 18
Die Hard, Action, John McTiernan, 1988, 132, 16
Mission Impossible Fallout, Action, Christopher McQuarrie, 2018, 147, 13
Top Gun Maverick, Action, Joseph Kosinski, 2022, 131, 10
The Avengers, Action, Joss Whedon, 2012, 143, 10
Casino Royale, Action, Martin Campbell, 2006, 144, 13
The Batman, Action, Matt Reeves, 2022, 176, 14
Indiana Jones and the Last Crusade, Adventure, Steven Spielberg, 1989, 127, 10
Jurassic Park, Adventure, Steven Spielberg, 1993, 127, 10
Jumanji, Adventure, Joe Johnston, 1995, 104, 6
Pirates of the Caribbean, Adventure, Gore Verbinski, 2003, 143, 10
The Revenant, Adventure, Alejandro González Iñárritu, 2015, 156, 16
Life of Pi, Adventure, Ang Lee, 2012, 127, 10
King Kong, Adventure, Peter Jackson, 2005, 187, 13
The Mummy, Adventure, Stephen Sommers, 1999, 124, 10
Uncharted, Adventure, Ruben Fleischer, 2022, 116, 10
Journey to the Center of the Earth, Adventure, Eric Brevig, 2008, 93, 6
Toy Story, Animation, John Lasseter, 1995, 81, 3
Finding Nemo, Animation, Andrew Stanton, 2003, 100, 3
Frozen, Animation, Chris Buck, 2013, 102, 3
Shrek, Animation, Andrew Adamson, 2001, 90, 6
Coco, Animation, Lee Unkrich, 2017, 105, 6
Up, Animation, Pete Docter, 2009, 96, 3
Cars, Animation, John Lasseter, 2006, 117, 3
Inside Out, Animation, Pete Docter, 2015, 95, 3
Ratatouille, Animation, Brad Bird, 2007, 111, 3

```

Questo file può essere modificato esclusivamente dai proiezionisti, i quali hanno la possibilità di aggiungere nuovi film alla lista disponibile nel sistema.

Questa banca dati risulta molto utile per consentire ai proiezionisti di selezionare il film associato a una nuova proiezione. Per ogni film vengono memorizzate diverse informazioni, tra cui:

- il titolo del film, univoco per ogni oggetto;
- il regista;
- l'anno di uscita;
- la durata in minuti;
- l'età minima consentita alla visione.

Il vincolo relativo all'età minima permette al sistema di controllare automaticamente l'accesso alle prenotazioni. Infatti, se l'età dell'utente registrato risulta inferiore all'età minima richiesta dal film, non sarà possibile prenotare posti per quella determinata proiezione.

8.4 Proiezioni.csv

```

data,genere,regista,anno,durata,eta,costo
"2027-12-30 10:30","A Beautiful Mind",Biography,"Ron Howard",2001,135,12,8.5
"2027-12-29 16:00","Good Will Hunting",Drama,"Gus Van Sant",1997,126,0,8.5
"2027-12-29 12:30","The Best Years of Our Lives",Drama,"William Wyler",1946,170,12,8.5
"2027-12-29 10:00","To Have and Have Not",Adventure,"Howard Hawks",1944,100,6,8.0
"2027-12-28 15:30","Blue Velvet",Drama,"David Lynch",1986,120,12,7.5
"2027-12-28 13:00","The Philadelphia Story",Comedy,"George Cukor",1940,112,12,8.5
"2027-12-28 10:00","Good Will Hunting",Drama,"Gus Van Sant",1997,126,0,6.5
"2027-12-27 14:00","The Blues Brothers",Action,"John Landis",1980,133,0,8.5
"2027-12-27 10:30","The Bridge on the River Kwai",Adventure,"David Lean",1957,161,6,7.5
"2027-12-26 19:30","Good Will Hunting",Drama,"Gus Van Sant",1997,126,0,10.0
"2027-12-26 16:00","The Bridge on the River Kwai",Adventure,"David Lean",1957,161,6,8.5
"2027-12-26 13:00","Good Will Hunting",Drama,"Gus Van Sant",1997,126,0,8.5
"2027-12-26 10:00","Blue Velvet",Drama,"David Lynch",1986,120,12,7.5
"2027-12-25 22:00","The Best Years of Our Lives",Drama,"William Wyler",1946,170,12,10.0
"2027-12-25 18:00","Gandhi",Biography,"Richard Attenborough",1982,191,0,9.0
"2027-12-25 14:30","The Lord of the Rings: The Fellowship of the Ring",Action,"Peter Jackson",2001,178,0,7.0
"2027-12-25 10:30","The Lord of the Rings: The Fellowship of the Ring",Action,"Peter Jackson",2001,178,0,8.0
"2027-12-24 13:30","Misery",Drama,"Rob Reiner",1990,107,16,8.0
"2027-12-24 10:30","The Treasure of the Sierra Madre",Adventure,"John Huston",1948,126,12,8.5
"2027-12-23 10:00","The Hurricane",Biography,"Norman Jewison",1999,146,16,8.5
"2027-12-22 17:00","C.R.A.Z.Y.",Comedy,"Jean-Marc Vallée",2005,129,12,8.0
"2027-12-22 14:30","Misery",Drama,"Rob Reiner",1990,107,16,7.5
"2027-12-22 10:30","Veer-Zaara",Drama,"Yash Chopra",2004,192,0,7.5
"2027-12-21 16:00","Primal Fear",Crime,"Gregory Hoblit",1996,129,12,7.0
"2027-12-21 13:00","The Blues Brothers",Action,"John Landis",1980,133,0,8.0

```

Questo file rappresenta la struttura più capiente ed essenziale per il corretto funzionamento dell'intero programma.

Al suo interno sono memorizzate tutte le proiezioni inserite dai proiezionisti, i quali hanno la possibilità di:

- aggiungere nuove proiezioni;
- modificare proiezioni esistenti;
- eliminare proiezioni già registrate.

Le operazioni di modifica ed eliminazione possono essere effettuate solamente nel caso in cui non siano già presenti prenotazioni associate a quella determinata proiezione.

Ogni proiezione è composta dalle informazioni relative a un film selezionato dall'apposito file correlato contenente l'elenco dei film disponibili.

Durante l'esecuzione del programma, il sistema mostra esclusivamente le proiezioni ancora disponibili, escludendo automaticamente quelle già scadute, ovvero le proiezioni con una data antecedente a quella corrente.

9. Scelte Algoritmiche

All'interno del progetto, la priorità è sempre stata quella di scegliere algoritmi poco complessi, facili da utilizzare e soprattutto da comprendere, senza trascurare le prestazioni del programma.

Infatti, in tutti gli algoritmi implementati si è evitato l'uso di soluzioni ricorsive, al fine di non appesantire lo stack durante l'esecuzione. Sono state invece adottate implementazioni iterative, che hanno migliorato sia la leggibilità del codice sia le prestazioni complessive del progetto.

Di seguito vengono presentati quattro algoritmi che rappresentano la filosofia appena descritta.

9.1 Cifratura di Password

```
public static String EncodedPsw(String Password){
    String encode="";

    for(int i=0;i<Password.length();i++){
        char c=Password.charAt(i);
        c=(char)(c*chiave);
        encode+=c;
    }
    return encode;
}
```

Il seguente algoritmo, utilizzato in fase di registrazione dell'utente, consente di cifrare la password in input tramite un semplice sistema a chiave privata. A partire da una chiave con valore noto solo al sistema, vengono applicate operazioni matematiche (in questo caso la moltiplicazione) ai caratteri della stringa della password, modificandoli in modo da renderli illeggibili. In questo modo si garantisce una maggiore protezione delle informazioni degli utenti che utilizzano la piattaforma.

9.2 Decifratura di Password

```

public static String DecodePsw(String Password){ /
    String decode="";

    for(int i=0;i<Password.length();i++){
        char c=Password.charAt(i);
        c=(char)(c/chiave);
        decode+=c;
    }

    return decode;
}

```

A differenza del codice precedente, questo algoritmo svolge l'operazione opposta: a partire dalla password cifrata, utilizzando la stessa chiave impiegata per la cifratura, e applicando l'operazione inversa della moltiplicazione (la divisione), è possibile ricostruire la password originale. Il processo avviene analizzando e trasformando ogni carattere della stringa cifrata.

9.3 Cambio Data Prenotazione

```

public static void NuovaData(Prenotazione p, int posti, Prenotazione nuova){
    Scanner input = new Scanner(System.in);
    int scelta;
    ArrayList<Proiezione> pro= leggiProiezioni();
    ArrayList<Proiezione> filtro = new ArrayList<>();
    for(Proiezione proiezione : pro)
        if(proiezione.GetFilm().getTitolo().equals(p.getProiezione().GetFilm().getTitolo()) && (posti + Cliente.PostiGiàPrenotati() <= proiezione.GetPostiDisponibili()))
            filtro.add(proiezione);
    if(filtro.isEmpty()){
        System.out.println("non ci sono proiezioni disponibili");
    }
    else {
        do {
            System.out.println("Seleziona la proiezione: \n");
            for (int i = 0; i < filtro.size(); i++) {
                System.out.println((i + 1) + " " + filtro.get(i).toString());
            }
            System.out.println("SCELTA: ");
            scelta = input.nextInt();
            input.nextLine();
        } while (scelta < 1 || scelta > filtro.size());
        if(filtro.get(scelta-1).GetData().isBefore(LocalDate.now()))
            System.out.println("Non è possibile cambiare data");
        else {
            nuova.getProiezione().SetData(filtro.get(scelta - 1).GetData());
            nuova.getProiezione().SetCosto(filtro.get(scelta - 1).GetCosto());
            nuova.getProiezione().SetOra(filtro.get(scelta - 1).GetOra());
        }
    }
}

```

Algoritmo più complesso rispetto ai precedenti, che permette al cliente di modificare la data della propria prenotazione.

Per prima cosa vengono ricercate altre proiezioni con le stesse caratteristiche, come il titolo del film, ma con data e orario diversi rispetto a quelli iniziali. L'utente può quindi scegliere una proiezione alternativa, mantenendo invariati i posti prenotati, se possibile.

La data della proiezione può essere modificata solo se non è già trascorsa.

Una volta selezionata la nuova proiezione, partendo dall'oggetto prenotazione dell'utente, vengono aggiornati i campi relativi a data, ora e costo in base ai dati della nuova proiezione scelta. Questa modifica è stata effettuata per permettere di risalire correttamente alla proiezione nel file dedicato.

9.4 Ricerca Proiezioni Disponibili

```
public ArrayList<Proiezione> ProiezioniDisponibili(){
    ArrayList<Proiezione> p = Proiezionista.leggiProiezioni();
    ArrayList<Proiezione> pro = new ArrayList<>();

    for(Proiezione proiezione : p){
        if(this.GetAnni()>=proiezione.GetFilm().getEtaMinima() && PostiGiaPrenotati(proiezione)<200)
            pro.add(proiezione);
    }
    return pro;
}
```

Questo algoritmo permette di filtrare tutte le proiezioni, restituendo una lista contenente quelle selezionabili dai clienti.

La classe Proiezione consente di recuperare tutte le proiezioni presenti all'interno dell'apposito file.

I vincoli del filtro sono due: il primo riguarda il numero di posti disponibili; se una proiezione ha già raggiunto il limite di 200 prenotazioni, non verrà inclusa nell'ArrayList. Il secondo vincolo riguarda l'età del cliente: se l'età del cliente è maggiore o uguale all'età minima richiesta dal film, la proiezione viene inserita nella lista filtrata; in caso contrario, non viene considerata.

10. Sitografia e Riferimenti

Javadoc

Javadoc è uno strumento di documentazione automatico utilizzato all'interno del progetto, che consente di descrivere in modo strutturato il comportamento delle classi e dei metodi, includendo informazioni su parametri, valori di ritorno e logica implementativa.

La documentazione viene generata a partire da commenti specifici inseriti direttamente nel codice sorgente (all'interno dei file presenti nella cartella src), scritti secondo una sintassi standard riconosciuta da Javadoc.

Nel progetto, la documentazione è stata generata tramite l'ambiente di sviluppo IntelliJ IDEA, che permette l'esportazione automatica dei file HTML corrispondenti.

Il risultato della generazione è un insieme di file in formato HTML consultabili tramite browser, organizzati in una struttura navigabile, e salvati all'interno della cartella doc del progetto. Questa documentazione facilita la comprensione del codice, migliorando la manutenibilità e la leggibilità del sistema.

IntelliJ IDEA

IntelliJ IDEA è l'ambiente di sviluppo integrato (IDE) utilizzato per la realizzazione del progetto CinemaX. Esso ha permesso la scrittura e l'organizzazione del codice sorgente, in particolare delle classi che compongono il sistema, fornendo strumenti avanzati per la gestione del progetto e lo sviluppo del software.

L'IDE è stato utilizzato per diverse attività fondamentali, tra cui:

- **Sviluppo del codice:** scrittura e gestione delle classi e dei relativi metodi.
- **Esecuzione dell'applicazione:** compilazione ed esecuzione del programma direttamente dall'ambiente di sviluppo.
- **Testing e debug:** individuazione e correzione di errori tramite strumenti di debug e analisi del flusso di esecuzione.
- **Generazione della documentazione Javadoc:** creazione automatica della documentazione HTML a partire dai commenti presenti nel codice sorgente.
- **Creazione dell'archivio JAR:** esportazione del progetto in un file eseguibile Java, utilizzato per la distribuzione dell'applicazione.

Il file JAR generato è stato salvato all'interno della cartella bin del progetto, rendendo possibile l'esecuzione del software in un ambiente Java senza necessità di ricompilazione.

Sitografia

La seguente sitografia raccoglie tutte le risorse online e i materiali di supporto consultati durante lo sviluppo del progetto *CinemaX* e la redazione del Manuale Tecnico e del Manuale Utente.

Le fonti includono documentazione ufficiale, guide pratiche, tutorial video e materiale didattico fornito dal docente.

1. Oracle Help Center

Utilizzato come riferimento principale per approfondire le funzionalità delle classi **LocalDate** e **LocalTime**.

La documentazione ha permesso di comprendere l'utilizzo corretto delle API moderne per la gestione delle date e degli orari, preferendole a soluzioni più datate o deprecate.

2. W3Schools

Consultato per:

- gestione dei file in Java
- utilizzo dei principali metodi della classe **ArrayList**
- manipolazione delle collezioni
- esempi pratici di lettura e scrittura su file

Le spiegazioni chiare e gli esempi immediati hanno facilitato l'implementazione delle funzionalità di base del progetto.

3. YouTube

Utilizzato come supporto pratico attraverso tutorial video relativi a:

- generazione della documentazione **Javadoc**
- creazione di archivi **JAR** tramite IntelliJ IDEA
- configurazione del progetto e procedure di esportazione

I video hanno fornito una guida visiva utile per comprendere passaggi operativi non sempre immediati.

4. Dispense e Slide del Docente

Le slide del corso (PPT) fornite dal professore sono state fondamentali per:

- definire la struttura del progetto
- seguire le linee guida per la creazione del **Manuale Tecnico**
- seguire le linee guida per la creazione del **Manuale Utente**

Questo materiale ha rappresentato il riferimento principale per garantire coerenza con gli obiettivi didattici del progetto.

5. Copilot (Microsoft)

Utilizzato per:

- generare **immagini simulate** dei terminali Linux e macOS
- mostrare l'output del comando `java -version`

Le immagini simulate sono state utili per rendere più chiara la procedura di verifica dell'installazione di Java sui diversi sistemi operativi, poiché non erano disponibili macchine fisiche o virtuali con sistemi operativi Linux e macOS durante la fase di documentazione.

- direttamente tramite IntelliJ IDEA. Le guide hanno facilitato la comprensione delle procedure di esportazione e documentazione del progetto.

11. Credenziali di Test

Per poter testare tutti i possibili utenti presenti nel programma cioè cliente, bigliettaio e proiezionista inserire le seguenti credenziali durante la fase di login:

USERNAME	PASSWORD
proiezionista1	ComoCinema1
proiezionista2	admin345
bigliettaio1	ComoCinema2
bigliettaio2	admin2404
bigliettaio3	An?110275
bigliettaio4	Fantasia88P
bigliettaio5	Luce+194
Alix2008 (Cliente)	An?110275